



Лекція 7

Реляційні БД Java DataBase Connectivity (JDBC) JPA та Hibernate

Визначення бази даних

База даних (БД) - це організована колекція даних, яка зберігається і обробляється комп'ютерною системою. Бази даних дозволяють зберігати, оновлювати, видаляти та виконувати пошук по інформації за допомогою різних запитів.

Роль баз даних

Роль баз даних у програмуванні - бази даних є ключовою складовою будь-якої програмної системи. Вони забезпечують збереження даних у відформатованому, структурованому вигляді, що полегшує доступ до цих даних і дозволяє програмістам ефективно працювати з інформацією. Бази даних також дозволяють забезпечити консистентність, цілісність та безпеку даних.

Види баз даних

Реляційні

- Цей тип баз даних використовує реляційну модель даних для організації даних у вигляді таблиць, які містять рядки та стовпці. Найпоширенішими реляційними базами даних є MySQL, PostgreSQL, Oracle, Microsoft SQL Server тощо.

Нереляційні бази даних (NoSQL)

- Цей тип баз даних використовує різноманітні моделі даних, такі як ключ-значення, документи, стовпчикові та графові бази даних. NoSQL бази даних зазвичай використовуються для розподілених систем та великих об'ємів даних. Приклади NoSQL баз даних включають MongoDB, Elasticsearch, Cassandra, Redis тощо.

Види баз даних

Графові бази даних

- Цей тип баз даних спеціалізується на зберіганні та обробці даних у вигляді графів. Графові бази даних корисні для аналізу взаємозв'язків між об'єктами та вирішення різноманітних задач, таких як соціальні мережі, маршрутизація мережі. Приклади графових баз даних включають Neo4j, Amazon Neptune, ArangoDB

Види баз даних

Сховище “ключ-значення”

- Цей тип баз даних використовує просту структуру, де дані зберігаються у вигляді пар ключ-значення. Такий підхід дозволяє швидко отримувати доступ до даних за допомогою ключа. Сховище "ключ-значення" ідеально підходить для швидкого доступу до даних, особливо в розподілених системах. Приклади: Riak, Amazon DynamoDB.

Види баз даних

Документо-орієнтоване сховище

- Цей тип баз даних дозволяє зберігати та опрацьовувати документи у вигляді JSON, XML або інших форматів. Кожен документ може мати різну структуру, що робить цей тип бази даних дуже гнучким. Документо-орієнтовані сховища ідеально підходять для сценаріїв, де структура даних може змінюватися чи де потрібно зберігати великі об'єми неструктурованих даних. Приклади: MongoDB, Elasticsearch, CouchDB.

Стовпчасте сховище

- Цей тип баз даних організує дані у вигляді стовпців замість рядків. Він оптимізований для ситуацій, коли необхідно працювати з великими обсягами даних, але частіше здійснювати доступ до окремих стовпців, а не всієї інформації з рядка. Приклади включають HBase, Cassandra.

Основні поняття РБД

Таблиця

Колонка

Рядок

Первинний ключ

Зовнішній ключ

STUDENTS			
ID	NAME	BIRTH_DATE	ID_GROUP
1	Юрко Богун	12.12.1995	21

Основні поняття РБД

Таблиця - основна структурна одиниця реляційної бази даних. Представляє собою двовимірну структуру, що складається з рядків та стовпців, де кожен рядок представляє запис, а кожен стовець - поле або атрибут.

Ключ - це стовець (може бути декілька стовпців), що додається до таблиці і дозволяє встановити зв'язок із записами в іншій таблиці.

Існують ключі двох типів: первинні і вторинні (зовнішні).

Первинний ключ - унікальний ідентифікатор кожного запису в таблиці. Зазвичай складається з одного або кількох стовпців, а його значення не може повторюватися у таблиці. Первинний ключ не допускає значень Null і завжди повинен мати унікальний індекс.

Зовнішній ключ - стовець або група стовпців у таблиці, які посилаються на ключ іншої таблиці. Використовується для встановлення зв'язку між двома таблицями.

Основні поняття РБД

Індекс - структура даних, яка прискорює швидкий доступ до даних у таблиці, забезпечуючи швидкий пошук та сортування.

Нормалізація - процес організації даних у реляційній базі даних з метою уникнення аномалій і забезпечення ефективного зберігання даних. Таблиці реляційної бази даних можуть бути нормалізовані до різного ступеня, щоб забезпечити оптимальну структуру даних.

Транзакція - логічна одиниця роботи з базою даних, яка складається з одного або декількох запитів, які виконуються як атомарна операція. Транзакції забезпечують цілісність даних та відновлюють базу даних у випадку збоїв.

Логічні відношення

- **один – до – одного (One-to-One)** – кожному запису з однієї таблиці відповідає один запис у іншій таблиці;
- **один – до – багатьох (One-to-Many)** – кожному запису з однієї таблиці відповідає кілька записів у іншій таблиці;
- **багато – до – багатьох (Many-to-Many)** – безлічі записів з однієї таблиці відповідає кілька записів в іншій таблиці. Цей тип зв'язку зазвичай потребує додаткової таблиці -посередника (зв'язкової таблиці), яка зберігає зв'язки між двома зв'язаними сутностями. Наприклад, один автор може мати багато книг, а кожна книга може мати багато авторів.

Логічне відношення (Many-to-Many)

Таблиця "Автори":

author_id	name
1	Author1
2	Author2
...	...

Таблиця "Автори_Книги":

author_book_id	author_id	book_id
1	1	101
2	1	102
3	2	101
...	...	...

Таблиця "Книги":

book_id	title
101	Book1
102	Book2
...	...

У таблиці "Автори_Книги" кожен запис представляє зв'язок між конкретним автором (через стовпець "author_id") та конкретною книгою (через стовпець "book_id").

Кожен запис має свій унікальний ідентифікатор "author_book_id".

Логічне відношення (Many-to-Many)

```
CREATE TABLE Authors (  
  author_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE Books (  
  book_id INT PRIMARY KEY AUTO_INCREMENT,  
  title VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE Authors_Books (  
  author_book_id INT PRIMARY KEY AUTO_INCREMENT,  
  author_id INT,  
  book_id INT,  
  FOREIGN KEY (author_id) REFERENCES Authors(author_id),  
  FOREIGN KEY (book_id) REFERENCES Books(book_id)  
);
```

Поля "author_id" та "book_id" у таблиці "Автори_Книги" будуть зовнішніми ключами, які посилаються на відповідні поля в таблицях "Автори" та "Книги" відповідно.

Індекси в РБД

Індекси в РБД призначені для оптимізації запитів SQL.

Як вони працюють: При створенні індексу, база даних побудує дерево або хеш-таблицю для швидкого пошуку значень. Запити, які використовують стовпець, що індексований, можуть швидко знаходити відповідні записи за значенням цього стовпця.

Коли використовувати індекси: Індекси корисні у випадках, коли ви часто виконуєте запити, які фільтрують, сортують або з'єднують дані за допомогою певних стовпців. Вони допомагають зменшити час виконання запитів, особливо при роботі з великими обсягами даних.

Попередження та обмеження: Створення багато індексів може призвести до збільшення об'єму бази даних та сповільнення часу вставки, оновлення та видалення даних. Тому важливо ретельно розглядати, які стовпці індексувати, а також регулярно аналізувати та оптимізувати індекси у вашій базі даних.

Індекси в РБД

Вибір індексу (умова відбору, сортування, селективність)

Умова відбору: Індекс слід створювати на тих стовпцях, які часто використовуються у WHERE, JOIN або ORDER BY запитих. Це допомагає оптимізувати пошук і вибірку рядків за певними умовами.

Сортування: Якщо запити часто сортуються за певними стовпцями, то створення індексу на цих стовпцях полегшує виконання таких запитів.

Селективність: Це відношення кількості унікальних значень у стовпці до загальної кількості рядків. Індеси більш ефективні, коли вони є селективними, тобто коли значення в індексованому стовпці є рідкісними. Якщо стовпець має низьку селективність, індекс може бути менш ефективним.

Індекси в РБД

Унікальні індекси

Унікальний індекс: гарантує унікальність значень у відповідному стовпці або комбінації стовпців. Вони використовуються для підтримки цілісності даних і забороняють дублікати значень.

```
CREATE UNIQUE INDEX email_unique_index ON users (email);
```

Ця команда створює унікальний індекс з назвою "**email_unique_index**" для стовпця "email" у таблиці "users". Після виконання цієї команди база даних гарантує, що значення у цьому стовпці будуть унікальними.

Індекси в РБД

Складові індекси (покриваючий індекс)

Складовий індекс - це індекс, що включає в себе стовпці, які використовуються у запиті, а також стовпці, які необхідні для вибірки даних без необхідності обходу всієї таблиці. Це дозволяє базі даних використовувати лише індекс, щоб задовольнити запит, що прискорює його виконання.

EXPLAIN

Команда EXPLAIN в SQL використовується для аналізу того, як база даних виконує конкретний запит. Вона показує план виконання запиту, включаючи індекси, які використовуються, методи з'єднання таблиць та інші деталі. Це допомагає виявити можливість оптимізації запитів за допомогою індексів.

Індекси в РБД

Недоліки індексів:

- Зайві індекси:** Велика кількість непотрібних індексів може сповільнювати вставку, оновлення та видалення даних, а також займати додаткове місце в базі даних.
- Періодична підтримка:** Індекси потребують періодичної підтримки, такої як перебудова або перестройка, особливо при частій модифікації даних.
- Погана селективність:** Індекси на стовпцях з низькою селективністю можуть бути менш ефективними, оскільки вони можуть повертати велику кількість рядків у вибірці, що призводить до більшого часу виконання запитів.

Створення індексів

Приклад 1:

Припустимо, у нас є таблиця "Users" з колонками "Id", "first_name" та "last_name". Щоб покращити продуктивність пошуку користувачів за їхнім прізвищем, ми можемо створити індекс на колонці "last_name".

```
CREATE INDEX idx_last_name ON Users (last_name);
```

Приклад 2:

У нас є таблиця "Products" з колонками "ID", "Name", "Category" та "Price". Ми хочемо знайти всі продукти у певній категорії, відсортовані за ціною. Для цього ми можемо створити складовий індекс на колонках "Category" та "Price".

```
CREATE INDEX idx_category_price ON Products (Category, Price);
```

Створення індексів (важливо)

Приклад 3:

Припустимо, у нас є таблиця "Users" з колонками "id", "first_name", "last_name" та "age". Ми створюємо "Складовий індекс" на три колонки "first_name", "last_name" та "age".

```
CREATE INDEX idx_last_name ON Users (first_name, last_name, age);
```

Робимо запит але фільтруємо за **last_name**.

```
SELECT u.first_name, u.last_name, u.age  
FROM Users u  
WHERE u.last_name = 'LastNameTest'
```

У даному випадку "Складовий індекс" не буде використовуватись. БД просто не може пропустити перший стовпець у складовому індексі.

Транзакції

Транзакція — представляє собою групу послідовних операцій з базою даних, які виконуються як єдина логічна одиниця роботи. Кожна транзакція має бути або виконана повністю і успішно, або не виконана зовсім.

ACID - вимоги до транзакційної системи

- **Atomicity** — Атомарність
- **Consistency** — Послідовність
- **Isolation** — Ізольованість
- **Durability** — Довговічність

Рівні ізоляції транзакцій: dirty read, read committed, repeatable read, serializable

JDBC (Java DataBase Connectivity) - платформо-незалежний промисловий стандарт взаємодії Java-додатків з різними СУБД, реалізований у вигляді пакета `java.sql`, що входить в склад Java

JDBC побудований по **драйверній архітектурі**

- Клієнт використовує інтерфейси JDBC API для доступу до даних
- Драйвер JDBC реалізує інтерфейси

Основні інтерфейси

DataSource - об'єкт, через який встановлюється з'єднання (Connection) з БД

Connection - з'єднання з БД - через нього виконуються всі операції по взаємодії з БД

Statement, PreparedStatement - інтерфейси для виконання запитів до БД

ResultSet - для видобування даних і набору даних, які повертає **SELECT**

Встановлення з'єднання з БД

// один із варіантів створення DataSource для MS SQL

```
static SQLServerDataSource createDatasource() {  
    // SQLServerDataSource є реалізацією інтерфейсу DataSource  
    SQLServerDataSource dataSource = new SQLServerDataSource();  
    dataSource.setUser("myusername");  
    dataSource.setPassword("mypassword");  
    dataSource.setServerName("localhost");  
    dataSource.setDatabaseName("stud");  
    return dataSource;  
}
```

```
DataSource dataSource = createDatasource();
```

```
try (Connection connection = dataSource.getConnection()) {  
    // робимо запити до БД  
}
```

Встановлення з'єднання з БД

```
public static void main(String[] args) {  
    // Підключення до бази даних  
    DataSource dataSource = MyDataSourceFactory.getDataSource(); // Отримуємо об'єкт DataSource  
  
    try (Connection connection = dataSource.getConnection()) { // Встановлюємо з'єднання  
        // Виконуємо запити до бази даних  
        Statement statement = connection.createStatement(); // Створюємо об'єкт Statement для виконання SQL-запитів  
        ResultSet resultSet = statement.executeQuery( sql: "SELECT * FROM users"); // Виконуємо SELECT-запит  
  
        // Обробляємо результати запиту  
        while (resultSet.next()) {  
            int id = resultSet.getInt( columnLabel: "id");  
            String name = resultSet.getString( columnLabel: "name");  
            int age = resultSet.getInt( columnLabel: "age");  
            System.out.println("User: id=" + id + ", name=" + name + ", age=" + age);  
        }  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

Встановлення з'єднання з БД

```
public class MyDataSourceFactory {  
  
    no usages    new *  
    public static DataSource getDataSource() {  
        BasicDataSource dataSource = new BasicDataSource();  
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");  
        dataSource.setUrl("jdbc:mysql://localhost:3306/mydatabase");  
        dataSource.setUsername("username");  
        dataSource.setPassword("password");  
        return dataSource;  
    }  
}
```

Методи виконання SQL-команд

Інтерфейс **Statement** надає три різні методи виконання SQL-команд:

executeQuery() - для запитів, результатом яких є один єдиний набір значень, таких як запити SELECT. Метод повертає набір даних, отриманий з бази

executeUpdate() - для виконання операторів INSERT, UPDATE або DELETE, а також для операторів DDL. Метод повертає ціле число, що показує, скільки рядків даних було модифіковано

execute() – виконує SQL-команди, які можуть повертати різноманітні результати. Наприклад, може використовуватися для операцій CREATE TABLE, і т.п.

Читаємо з БД

```
try (Connection connection = dataSource.getConnection()) {  
    Statement statement = connection.createStatement();  
    ResultSet rs = statement.executeQuery("select * from student");  
    while (rs.next()) {  
        System.out.println(getStudent(rs));  
    }  
}  
  
private static Student getStudent(ResultSet rs) throws SQLException {  
    Student result = new Student();  
    result.setId(rs.getLong("id"));  
    result.setName(rs.getString("name"));  
    result.setGroupId(rs.getLong("id_group"));  
    return result;  
}
```

Інтерфейс PreparedStatement

Екземпляри класів, що реалізують інтерфейс `PreparedStatement`, зберігають прекомпільовані SQL-вирази

Особливістю SQL-виразів в `PreparedStatement` є те, що вони можуть мати **параметри**

- Параметризовані вирази включають знаки питання в своєму тексті.
 - Наприклад: `SELECT name FROM persons ps WHERE age=?`
- Перед виконанням запиту значення кожного знака питання явно встановлюється методами `setXxx()`
 - Наприклад: `ps.setInt(1, 30);`

Використання `PreparedStatement` приводить до більш швидкого виконання запитів при їх **багаторазовому виклику** з різними значеннями параметрів

Приклад використання PreparedStatement

// Підготувати запит з параметрами

```
PreparedStatement st = connection.prepareStatement(  
    "SELECT * FROM student WHERE id > ? AND name = ?");
```

// Встановити значення параметрів

```
st.setInt(1, 100);  
st.setString(2, "ОЛЕГ");
```

// Виконати підготовлений запит

```
ResultSet rs = st.executeQuery();
```

Пакетне виконання запитів

Для виконання **набору із декількох запитів** на оновлення даних в інтерфейсі Statement є методи:

- **void** addBatch(String)
- **int[]** executeBatch()

Пакетне виконання запитів зменшує трафік між клієнтом та СУБД і може призвести до значного підвищення продуктивності

Пакетне виконання запитів

```
Statement stmt = connection.createStatement();  
stmt.addBatch(  
    "INSERT INTO student VALUES (2, 'Максим Коваль', 2)");  
stmt.addBatch(  
    "INSERT INTO student VALUES (3, 'Іван Шевченко', 3)");  
int [] updateCounts = stmt.executeBatch();  
stmt.close();
```

Робота з метаданими

Інформація про метадані (тобто про саму структуру, а не про зміст бази даних) буває двох видів:

- метадані **набору даних** (`ResultSet`)
- метадані **джерел даних**

Для представлення **метаданих набору даних** використовується інтерфейс `ResultSetMetaData`, який може бути отриманий наступним чином:

```
ResultSet rs = stmt.executeQuery(sqlString);  
ResultSetMetaData rsmd = rs.getMetaData();
```

Для отримання **метаданих бази даних** потрібно використовувати інтерфейс `DatabaseMetaData`. Він доступний за допомогою методу `getMetadata()` інтерфейса `Connection`:

```
DatabaseMetaData dbmd = connection.getMetadata();
```

`DatabaseMetaData` містить методи отримання інформації про структуру таблиць, атрибутів стовпців, про підтримку СУБД різноманітних функцій, і т.п.

Робота з метаданими (ResultSetMetaData)

```
public class ResultSetMetadataExample {  
  
    new *  
    public static void main(String[] args) {  
        // Підключення до бази даних  
        String url = "jdbc:mysql://localhost:3306/mydatabase";  
        String username = "username";  
        String password = "password";  
  
        try (Connection connection = DriverManager.getConnection(url, username, password)) {  
            // Виконуємо запит до бази даних  
            String sql = "SELECT id, name, age FROM users";  
            Statement statement = connection.createStatement();  
            ResultSet resultSet = statement.executeQuery(sql);  
  
            // Отримання метаданих набору даних  
            ResultSetMetaData rsmd = resultSet.getMetaData();  
  
            // Вивід метаданих  
            System.out.println("Number of columns: " + rsmd.getColumnCount());  
            for (int i = 1; i <= rsmd.getColumnCount(); i++) {  
                System.out.println("Column " + i + ":" );  
                System.out.println("    Name: " + rsmd.getColumnName(i));  
                System.out.println("    Type: " + rsmd.getColumnTypeName(i));  
                System.out.println("    Size: " + rsmd.getColumnDisplaySize(i));  
            }  
        } catch (SQLException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Робота з метаданими (DatabaseMetaData)

```
public class DatabaseMetadataExample {  
    new *  
    public static void main(String[] args) {  
        // Підключення до бази даних  
        String url = "jdbc:mysql://localhost:3306/mydatabase";  
        String username = "username";  
        String password = "password";  
  
        try (Connection connection = DriverManager.getConnection(url, username, password)) {  
            // Отримання метаданих бази даних  
            DatabaseMetaData dbmd = connection.getMetaData();  
  
            // Виводимо деяку інформацію про базу даних  
            System.out.println("Database Product Name: " + dbmd.getDatabaseProductName());  
            System.out.println("Database Product Version: " + dbmd.getDatabaseProductVersion());  
            System.out.println("Driver Name: " + dbmd.getDriverName());  
            System.out.println("Driver Version: " + dbmd.getDriverVersion());  
            System.out.println("URL: " + dbmd.getURL());  
            System.out.println("Username: " + dbmd.getUserName());  
  
            // Тут можна отримувати більше інформації про базу даних, таку як таблиці, стовпці, обмеження і т.д.  
            // Наприклад, можна вивести список таблиць  
            ResultSet tables = dbmd.getTables( catalog: null, schemaPattern: null, tableNamePattern: "%", types: null);  
            System.out.println("Tables:");  
            while (tables.next()) {  
                System.out.println("    " + tables.getString( columnLabel: "TABLE_NAME"));  
            }  
        } catch (SQLException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Робота з пулом з'єднань

Використання **пулів з'єднань** дозволяє підвищити продуктивність застосунку за рахунок перевикористання об'єктів-з'єднань з БД

```
public DataSource configureDataSource() {  
    HikariConfig config = new HikariConfig();  
    config.setDataSourceClassName("com.microsoft.sqlserver.jdbc.SQLServerDataSource");  
    config.addDataSourceProperty("serverName", "localhost");  
    config.addDataSourceProperty("portNumber", "1433");  
    config.addDataSourceProperty("databaseName", "stud");  
    config.addDataSourceProperty("user", "myusername");  
    config.addDataSourceProperty("password", "mypassword");  
    config.setMaximumPoolSize(20); // розмір пулу  
    return new HikariDataSource(config);  
}
```

<https://www.baeldung.com/hikaricp>

Дизайн застосунку: шар DAO

DAO (Data Access Objects) - шар (пласт) застосунку, відповідальний за взаємодію з персистентним сховищем даних.

```
public class StudentDao {  
    private final DataSource dataSource;  
    public StudentDao(DataSource dataSource) { this.dataSource = dataSource; }  
  
    public Student getById(long id) {  
        try (Connection connection = dataSource.getConnection()) {  
            // prepare statement, execute query, convert to Student  
        } catch (SQLException e) {  
            throw new RuntimeException("Error loading stud with id " + id, e);  
        }  
    }  
}
```



Дизайн застосунку: шар DAO

Бізнес-логіка: DAO повинен відокремлюватись від бізнес-логіки застосунку. Він не повинен містити складних алгоритмів або логіки, пов'язаної з правилами бізнесу.

Відображення користувацького інтерфейсу: DAO не повинен містити жодного коду, пов'язаного з відображенням інформації для користувача. Він має працювати на рівні доступу до даних.

Приклад додатку: pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>com.zaxxer</groupId>
  <artifactId>HikariCP</artifactId>
  <version>5.0.0</version>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.22</version>
  <scope>provided</scope>
</dependency>
```

Приклад додатку: application.yaml

spring:

datasource:

type: com.zaxxer.hikari.HikariDataSource

url: jdbc:mysql://localhost:3307/stud?useSSL=false

username: root

password: root

hikari:

maximum-pool-size: 5

connection-timeout: 60000

Приклад додатку: StudentDao (вибірка)

@Repository

@RequiredArgsConstructor

```
public class StudentDao {
```

```
    private final NamedParameterJdbcTemplate jdbcTemplate;
```

```
    public List<StudentData> listAllStudents() {  
        return jdbcTemplate.query("SELECT * FROM student", (rs, rowNum) -> {  
            StudentData result = new StudentData();  
            result.setId(rs.getLong("id"));  
            result.setFirstName(rs.getString("first_name"));  
            result.setLastName(rs.getString("last_name"));  
            return result;  
        });  
    }  
  
    ...  
}
```

Висновки про JDBC

Набір операцій JDBC стандартний та дуже простий:

- підключилися до бази даних;
- створили **statement**;
- виконали SQL-запит та отримали **Result set**;
- пройшлися циклом та отримали всі дані, які витягли з БД, якщо робили запит **select**.

Висновки про JDBC

Варто пам'ятати:

- Для кращої ефективності та безпеки, рекомендується використовувати підготовлені (PreparedStatement) замість звичайних стейтментів (Statement), а також використовувати пул з'єднань (Connection Pooling), наприклад, HikariCP.
- Пам'ятайте про безпеку: уникайте SQL-ін'єкцій шляхом використання параметризованих запитів та інших механізмів захисту.
- Завжди закривайте з'єднання та інші ресурси після завершення роботи з ними, щоб уникнути витоку ресурсів.

JPA i Hibernate



Проблеми взаємодії реляційної і об'єктної моделей даних

СУБД дає нам інформацію в **табличному форматі**, в той час, як Java оперує інформацією у вигляді певного **графа об'єктів**

- **Наслідування**

- В реляційній моделі немає ніякого поняття, схожого на наслідування, яке являється одним із ключових принципів ООП

- **Ідентифікація**

- Для БД є тільки одна сутність, за якою об'єкт може бути ідентифікований – це Primary Key.
У Java об'єкти можуть бути порівнювані за посиланням (`entity1 == entity2`) або за їх вмістом (`object1.equals(object2)`), що надає більше гнучкості у визначенні та роботі з ідентифікацією об'єктів.

- **Асоціація**

- В Java ми використовуємо посилання на об'єкти для асоціації, а в реляційній моделі – Foreign Key.

- **Доступ**

- В Java і в реляційній БД абсолютно різні способи отримання доступу до даних

Див. також https://en.wikipedia.org/wiki/Object-relational_impedance_mismatch

Object-Relational Mapping (ORM)

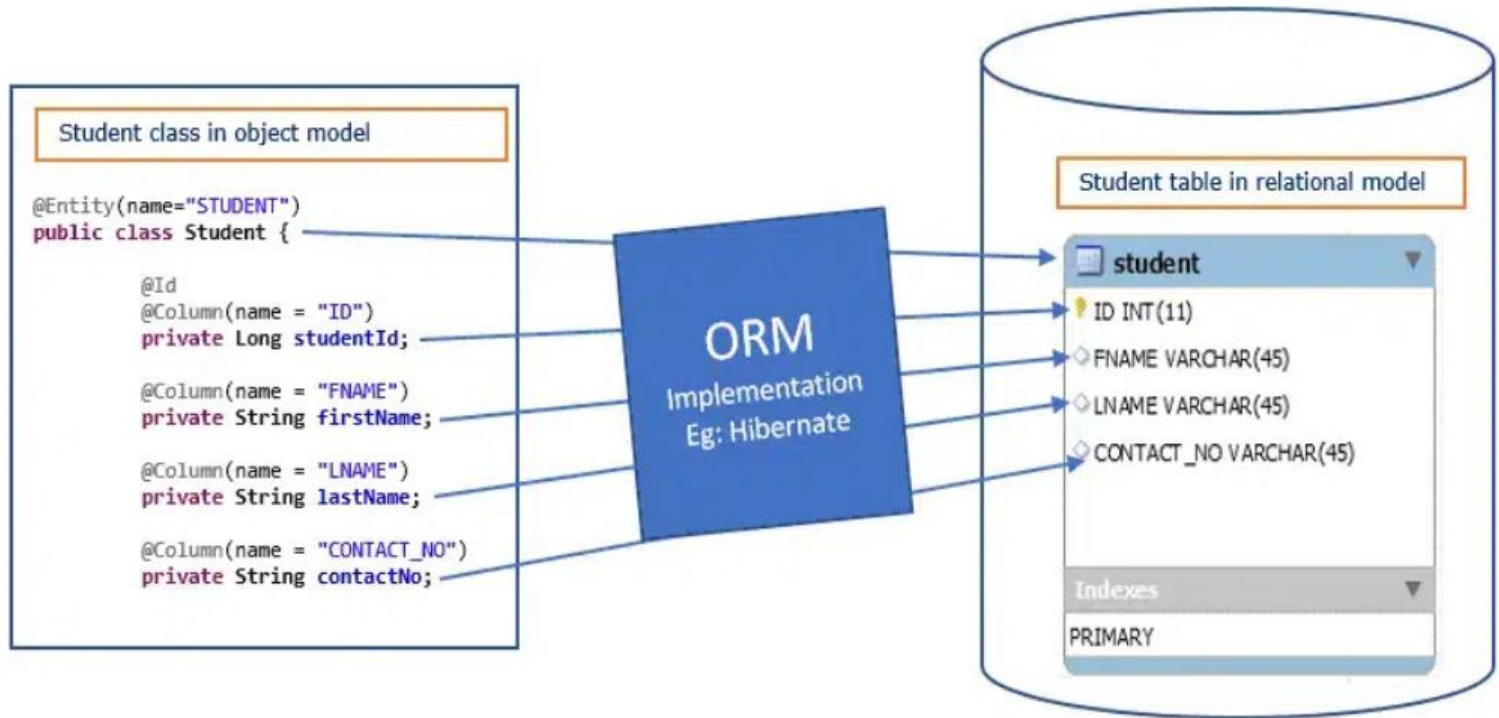
Засоби ORM дозволяють **проектувати об'єкти застосунку на таблиці** реляційної бази даних за допомогою **метаданих**, що описують співвідношення між об'єктами і таблицями



Базові операції з БД характерні для будь-якої інформаційної системи:

- Збереження об'єкта і його властивостей в БД
- Пошук об'єкта в БД по ID і його відновлення в java-об'єкт
- Видалення об'єкта з БД
- Завантаження списку об'єктів (пошук)

Object-Relational Mapping (ORM)



JDBC vs ORM

JDBC

- + Проста і зрозуміла обробка SQL-запитів
- + Зручна для невеликих застосунків
- + Простий і зрозумілий синтаксис
- + Взаємодія з БД і виконання SQL-запитів повністю контролюється розробником
- Розробник оперує поняттями БД, а не класами/об'єктами Java
- Потрібне знання SQL і реляційної моделі
- Складно використовувати і підтримувати у великих проектах
- Велика кількість кода

Переваги використання **ORM**

- + Дозволяє нашим бізнес методам звертатися не до БД, а до Java-класів
- + Пришвидшує розробку застосунку
- + Приховує від розробника SQL-запити
- + Дозволяє не думати про реалізації БД
- + Великий плюс ORM - ви пишете бізнес-логіку, а не SQL
- Продуктивність, проблеми з складними запитам, потребує вивчення

JPA и Hibernate

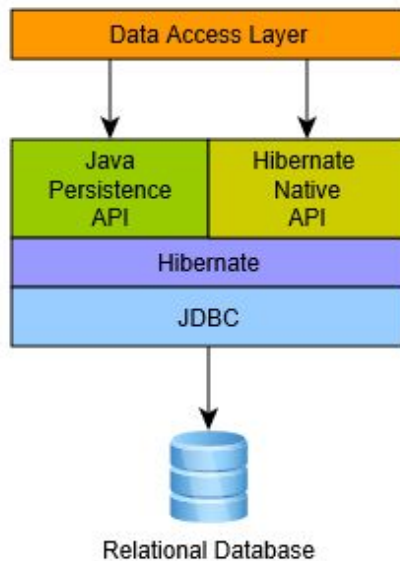
JPA (Java Persistence API) це специфікація Java EE і Java SE, що описує систему керування збереженням java об'єктів у таблиці реляційних баз даних.

- Java не містить реалізації JPA, але існує багато реалізацій даної специфікації від різних компаній (відкритих і ні).
- Це не єдиний спосіб збереження java об'єктів в бази даних (ORM систем), але один із самих популярних в Java

Hibernate одна із самих популярних відкритих реалізацій специфікації.

- JPA тільки описує правила і API, а Hibernate реалізовує описане.
- У Hibernate (як і в багатьох інших реалізацій JPA) є додаткові можливості, не описані в JPA (і не перенесені на інші реалізації JPA).

Архітектура Hibernate



Див. https://docs.jboss.org/hibernate/orm/5.4/userguide/html_single/Hibernate_User_Guide.html

Способи конфігурації сутностей і відношень

XML-знаходиться - в файлах **.hbm.xml*

```
<hibernate-mapping package="ua.profitsoft.ajd.lk4hiber">
  <class name="Student" table="student">
    <id name="id" column="id" type="long" unsaved-value="-1">
      <generator class="native"/>
    </id>
    <property name="name" />
  </class>
  ...
</hibernate-mapping>
```

За допомогою анотацій (із специфікації JPA)

```
@Entity(name = "student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

Hibernate + Spring

- Легка конфігурація
- Dependency Injection
- Декларативне управління транзакціями

Створюємо Spring Boot застосунок

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>com.zaxxer</groupId>
  <artifactId>HikariCP</artifactId>
  <version>5.0.0</version>
  <scope>provided</scope>
</dependency>
```

Application

@SpringBootApplication

@EnableTransactionManagement

public class LkHibernateApplication **implements** CommandLineRunner {

public static void main(String[] args) {

 SpringApplication.run(LkHibernateApplication.**class**, args);

}

@Override

public void run(String... args) **throws** Exception {

// we will add out code here

}

}

application.yaml

конфігурація DataSource

spring:

datasource:

type: com.zaxxer.hikari.HikariDataSource

url: jdbc:mysql://localhost:3307/stud?useSSL=false

username: root

password: root

hikari:

maximum-pool-size: 5

connection-timeout: 60000

конфігурація Hibernate

jpa:

hibernate:

dialect: org.hibernate.dialect.MySQLInnoDBDialect

ddl-auto: none (Каже щоб hibernate не створював/видаляв таблиці за замовчуванням)

show-sql: true

Data-об'єкт

```
@Entity(name = "Student")
@Getter @Setter // анотації із Lombok
@NoArgsConstructor
@ToString

public class StudentData {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private LocalDate birthDate;

    public StudentData(String name) {
        this.name = name;
    }
}
```

DAO

@Repository

@AllArgsConstructor

```
public class StudentDao {

    private EntityManager entityManager;

    public StudentData getStudent(long id) {
        return entityManager.find(StudentData.class, id);
    }

    public List<StudentData> listAll() {
        // запрос за помощью JPQL
        return entityManager.createQuery("SELECT s from Student s")
            .getResultList();
    }

    public void save(StudentData stud) {
        entityManager.persist(stud);
    }
}
```

Service

@Service

@AllArgsConstructor

```
public class StudentServiceImpl implements StudentService {
```

```
    private StudentDao studentDao;
```

@Override

```
    public List<StudentData> listAll() {
```

```
        return studentDao.listAll();
```

```
    }
```

@Transactional

```
    public void saveStudent(StudentData stud) {
```

```
        studentDao.save(stud);
```

```
    }
```

```
}
```

Запускаемо все

@SpringBootApplication

```
public class Lk8HibernateApplication implements CommandLineRunner {
```

@Autowired

```
private StudentService studentService;
```

```
public static void main(String[] args) {
```

```
    SpringApplication.run(Lk8HibernateApplication.class, args);
```

```
}
```

@Override

```
public void run(String... args) throws Exception {
```

```
    StudentData stud = new StudentData("Ihor");
```

```
    stud.setBirthDate(LocalDate.parse("2000-02-05", DateTimeFormatter.ISO_LOCAL_DATE));
```

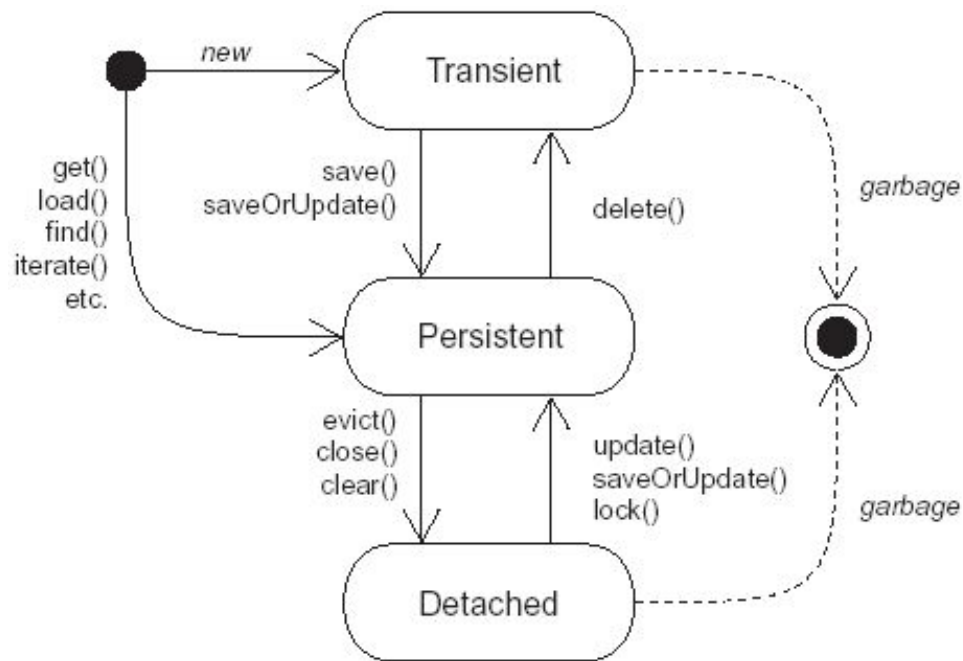
```
    studentService.saveStudent(stud);
```

```
    System.out.println("All students: " + studentService.listAll());
```

```
}
```

```
}
```

ЖИТТЄВИЙ ЦИКЛ ОБ'ЄКТІВ



Primary Key

При описі таблиці необхідно вказувати поле (поля), які являються **первинним ключем**

При використанні Hibernate завжди рекомендується використовувати **штучні ключі**

Для штучних ключів необхідно задавати генератор (є різні стратегії генерації)

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;
```

Вкладені об'єкти

@Embeddable

@NoArgsConstructor

@AllArgsConstructor

```
public class Address {  
    private String town;  
    private String street;  
    private String house;  
}
```

```
public class StudentData {  
    ...  
    @Embedded  
    @AttributeOverrides({  
        @AttributeOverride(name = "town", column = @Column(name = "address_town")),  
        @AttributeOverride(name = "street", column = @Column(name = "address_street")),  
        @AttributeOverride(name = "house", column = @Column(name = "address_house"))  
    })  
    private Address address;  
}
```

Many-to-one

```
@Entity(name = "Student")
@Getter
@Setter
@NoArgsConstructor
@ToString
@Table(name = "student_data") // Вказуємо назву таблиці в базі даних
public class StudentData {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name="id_group")
    private GroupData group;

    ...
}
```

One-to-many

```
@Entity(name = "Group")
@Getter @Setter // анотації із Lombok
@NoArgsConstructor
@ToString
@Table(name="group_data")
public class GroupData {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToMany(mappedBy="group")
    private Set<StudentData> students;

    ...
}
```

Типи завантаження

Не лінива - @Lazy(false)

При завантаженні із БД об'єкту відразу завантажуються всі об'єкти, на які посилається цей об'єкт (тобто які зберігаються у нього в колекціях)

Лінива - @Lazy(true)

Таке завантаження відбувається тільки при зверненні до відповідного об'єкту

Каскадні операції

```
@Entity(name = "Student")
```

```
...
```

```
public class StudentData {
```

```
    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
```

```
    @JoinColumn(name="id_student")
```

```
    private Set<PhoneNumber> phoneNumbers = new HashSet<>();
```

```
}
```

Залежна сутність при каскадних операціях

```
@Entity(name="PhoneNumber")
```

```
@Data
```

```
@NoArgsConstructor
```

```
@ToString
```

```
public class PhoneNumber {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private PhoneType type;
```

```
    private String number;
```

```
    public PhoneNumber(PhoneType type, String number) {
```

```
        this.type = type;
```

```
        this.number = number;
```

```
    }
```

```
}
```

Види каскадних операцій

```
package javax.persistence;
```

```
public enum CascadeType {
```

```
    /** Всі операції */
```

```
    ALL,
```

```
    /** Збереження */
```

```
    PERSIST,
```

```
    /** Об'єднання */
```

```
    MERGE,
```

```
    /** Видалення */
```

```
    REMOVE,
```

```
    /** Оновлення з БД */
```

```
    REFRESH,
```

```
    /** Від'єднання */
```

```
    DETACH
```

```
}
```

Запити за допомогою JPQL

```
public class StudentDao {  
    ...  
    public List<StudentData> listByGroup(GroupData group) {  
        return entityManager.createQuery("SELECT s FROM Student s " +  
            "WHERE s.group=:group")  
            .setParameter("group", group)  
            .getResultList();  
    }  
}
```

Запити за допомогою CriteriaBuilder

```
private SessionFactory sessionFactory;
```

```
public List<StudentData> listByName(String name) {  
    Session session = sessionFactory.openSession();  
    CriteriaBuilder cb = session.getCriteriaBuilder();  
    CriteriaQuery<StudentData> cr = cb.createQuery(StudentData.class);  
    Root<StudentData> root = cr.from(StudentData.class);  
    cr.select(root);  
    if (name != null) {  
        cr.where(cb.equal(root.get("firstName"), name));  
    }  
    return session.createQuery(cr).getResultList();  
}
```

Більш детально: <https://www.baeldung.com/hibernate-criteria-queries>



Дякую за увагу!